FSTT Local AI Assistant

1. Project overview

This project implements a local AI assistant using Python, Flask, Ollama, and locally stored course documents. The application answers course questions using retrieval-augmented generation (RAG), keeps conversation history for follow-up questions, supports structured JSON responses, and measures response time. All model inference is performed locally.

2. Application architecture and validation

Browser → Flask application → document retriever and conversation history → Ollama REST API → local language model → browser response.

- RAG test: The assistant explained reinforcement learning using course.txt and

reinforcement_learning.txt (62.84 s; 6.1 tokens/s).

- Memory test: A follow-up question comparing reinforcement learning and supervised learning was

answered correctly (12.72 s; 5.8 tokens/s).

- Structured response test: A deep-learning summary returned a JSON object with topic, summary,

key_points, and difficulty (29.8 s; 5.7 tokens/s).

- Missing information test: For quantum computing, the assistant returned “Information not available.”

with no retrieved sources (6.65 s; 7.8 tokens/s).

3. Report questions

Q1. What is Local AI?

Local AI means running artificial intelligence models directly on a personal computer without depending on cloud servers.

Q2. Difference between a Cloud LLM and a Local LLM?

A Cloud LLM runs on remote servers and usually requires an internet connection. A Local LLM runs on your own computer and can work offline.

Q3. What role does Ollama play?

Ollama allows us to download, manage, and run language models locally. It also provides an API to connect these models to Python applications.

Q4. What is model quantization?

Quantization reduces the numerical precision of model weights to decrease memory usage and improve performance, sometimes with a small loss in accuracy.

Q5. Why does model size affect memory consumption?

Larger models usually contain more parameters, which require more RAM or GPU memory during execution.

Q6. Why can a smaller model sometimes be preferable?

Smaller models generally require less memory, run faster, and work better on computers with limited hardware resources.

Q7. Advantages of local inference?
Local inference improves data privacy, allows offline use, avoids per-request cloud API costs, and provides greater control over the model.

Q8. Disadvantages of local inference?
Local inference requires sufficient hardware resources. Models can be slower, consume significant RAM, and require installation and maintenance.

Q9. Explain the architecture of your Python application.
My application uses Flask for the web interface, Python for processing requests, RAG to retrieve information from local documents, and Ollama to generate answers. It also maintains conversation history and measures response time.

Q10. Explain the role of the REST API.
The REST API allows my Python application to communicate with Ollama using HTTP requests and receive generated responses in JSON format.

Q11. What is RAG?
RAG (Retrieval-Augmented Generation) retrieves relevant information from documents and provides it to the language model to generate more accurate, context-based answers.

Q12. Difference between an LLM and a multimodal LLM?
A traditional LLM mainly processes text, while a multimodal LLM can process different types of information, such as text and images. In my experiment, Gemma 3 analyzed an image.

Q13. Compare two local models experimentally.
I compared Qwen3.5:4b and Gemma3:4b using the same prompt. Gemma3:4b was faster and used slightly less memory. The measured results are shown in the table below. Answer quality was not formally evaluated.

Metric	Qwen3.5:4b	Gemma3:4b
Reported disk size	6.55 GB	3.35 GB
Loaded size	3.20 GB	2.93 GB
VRAM	0.10 GB	0.11 GB
Load time	44.45 s	37.75 s
Response time	181.66 s	134.60 s
Generated tokens	803	801
Generation speed	6.00 tokens/s	8.55 tokens/s
System RAM used	86.6%	84.0%

Note: The reported 6.55 GB disk size for Qwen3.5:4b may correspond to another installed Qwen variant; verify using

ollama list. Quality scores were not measured.

Q14. Which model would you deploy on a computer with limited RAM? Explain.
I would choose Gemma3:4b because it used less system RAM (84.0% versus 86.6%) and generated text faster (8.55 versus 6.00 tokens/second). It is more suitable for my computer with 16 GB RAM.

Q15. What security risks exist when exposing a local LLM server to a network?
Exposing Ollama to a network can allow unauthorized users to access the model because Ollama has no authentication by default. Attackers could consume computing resources, access exposed data, or use malicious documents for prompt injection. I would use a firewall, authentication, HTTPS through a reverse proxy, and access restrictions for trusted users.

4. Benchmark discussion

On the tested 16 GB RAM computer, Gemma3:4b generated 8.55 tokens/s compared with 6.00 tokens/s for Qwen3.5:4b. It also had a shorter response time (134.60 s versus 181.66 s) and slightly lower reported system RAM usage (84.0% versus 86.6%). These results support selecting Gemma3:4b for this machine based on measured speed and resource use, not on untested quality claims.

5. Conclusion

The local assistant successfully answered questions from course documents, handled follow-up questions, generated structured JSON, and declined to answer when relevant information was unavailable. The experiments demonstrate that a practical local AI assistant can run on a 16 GB RAM computer, although response time and memory remain important constraints.